

Enci/ VENTAS

Documentación API / Swagger

Guía técnica de consulta, autorización y prueba de endpoints del MVP comercial

Versión 1.0

Proyecto Enci / Enci Ventas

Aplicación de Automatización de Fuerza de Ventas

Julio 2026

API REST FastAPI documentado	SWAGGER Uso controlado	FIREBASE Bearer JWT	PLM UTEM Entrega técnica
---------------------------------	---------------------------	------------------------	-----------------------------

Responsable de la documentación
Documento elaborado por el Grupo PLM de la Universidad Tecnológica Metropolitana (UTEM). Líder: Joshua Flores. Integrantes: Falon Berrios, Caroline Palacios y Eliseo Poblete.

Naturaleza del documento
Esta guía describe el acceso técnico a Swagger/OpenAPI, la forma correcta de autenticar pruebas con Firebase ID Token, el inventario de endpoints principales, los contratos de entrada y salida, la matriz de errores esperada y los criterios mínimos para validar la API antes de UAT o go-live. No habilita exposición pública de Swagger en producción.

Tabla de Contenidos

1. Introducción
2. Alcance y principios de uso
3. Acceso a Swagger/OpenAPI
4. Autenticación y autorización
5. Inventario general de endpoints
6. Contratos de entrada y salida
7. Pruebas funcionales desde Swagger
8. Contrato de errores HTTP
9. Seguridad de la consola Swagger
10. Validación técnica previa a entrega
11. Criterios de aceptación
12. Anexo de comandos y ejemplos

1. Introducción

La API de Enci/ Ventas constituye la capa de negocio central del MVP comercial. Su propósito es recibir solicitudes desde el frontend web, validar identidad mediante Firebase, aplicar reglas de autorización por rol y ejecutar operaciones sobre Firestore sin exponer la base de datos directamente al cliente.

La documentación Swagger/OpenAPI se utiliza como herramienta de validación técnica en ambientes controlados. Permite revisar rutas, parámetros, modelos, respuestas y códigos de error de manera estructurada. En producción, Swagger no debe operar como interfaz pública ni como canal de uso funcional del sistema.

Este documento está orientado a equipos técnicos, administradores del proyecto y responsables de recepción técnica que necesiten validar la API antes de una prueba UAT o antes de una salida controlada.

1.1 Objetivo del documento

- Describir el acceso permitido a la consola Swagger/OpenAPI.
- Formalizar la forma correcta de autenticar solicitudes protegidas.
- Consolidar la matriz de endpoints principales del MVP web avanzado.
- Identificar roles mínimos requeridos por módulo.
- Establecer contratos de error y criterios de validación técnica.
- Separar uso de desarrollo/UAT respecto de uso productivo.

1.2 Base técnica considerada

Fuente interna	Uso en esta guía
Repositorio oficial Enci	Referencia principal del backend FastAPI, frontend React/Vite y documentación técnica versionada.
Backend/app/main.py	Registro de routers, seguridad transversal, CORS, readiness y endpoints base.
Backend/app/docs.py	Consola Swagger personalizada con login Firebase para ambientes no productivos.
Backend/app/api/*.py	Definición de rutas por módulo: autenticación, clientes, comercial, usuarios, dashboard, competencia y RAG.
Backend/app/models/*.py	Contratos Pydantic, validaciones de payload, enums, límites y campos permitidos.
Criterios de aceptación y cierre	Criterio superior para interpretar MVP web, RAG parcial y aplicación Flutter como componente externo/parcial.

2. Alcance y principios de uso

La documentación API cubre la capa backend del sistema y su interacción con consumidores autorizados. El consumidor principal del MVP es el frontend web React/Vite. La app móvil Flutter parcialmente realizada puede usar la misma API cuando la integración corresponda, pero la existencia de la API no garantiza cobertura completa desde Flutter ni publicación de dicha app en tiendas.

Incluido	Fuera de alcance
Consulta y prueba de endpoints desde Swagger en ambiente local, development o UAT.	Exposición pública de Swagger en producción.
Autenticación Bearer con ID Token Firebase.	Uso de claves privadas o service accounts desde navegador.

Incluido	Fuera de alcance
Validación de roles admin, supervisor y vendedor.	Modificación manual de permisos fuera de los flujos definidos.
Pruebas de clientes, interacciones, oportunidades, propuestas, dashboards, usuarios y RAG.	Garantía de SLA externo del proveedor LLM o infraestructura no controlada.
Revisión de respuestas, errores, payloads y seguridad mínima.	Pruebas de carga avanzadas o auditoría ofensiva completa.

Principio operativo

Toda operación protegida debe ejecutarse con token Firebase válido. El frontend puede ocultar acciones por rol, pero el control obligatorio reside en backend. Firestore no debe quedar expuesto como canal directo de escritura desde cliente.

3. Acceso a Swagger/OpenAPI

En ambiente local, la consola de pruebas se accede desde el backend FastAPI levantado en el puerto 8000. La ruta de uso técnico es:

`http://127.0.0.1:8000/docs`
`http://localhost:8000/docs`

La consola utiliza una experiencia de prueba controlada: inicia sesión con Google, obtiene un ID Token Firebase y aplica el token al esquema HTTPBearer de Swagger para ejecutar endpoints protegidos.

Ruta	Uso	Ambiente permitido	Criterio
GET /docs	Consola Swagger personalizada	Local, development, UAT	Debe estar bloqueada o no expuesta en producción.
GET /openapi.json	Esquema OpenAPI consumido por Swagger	Local, development, UAT	En producción debe quedar inaccesible.
GET /docs/guia-presentacion-frontend.pdf	Guía auxiliar de demostración frontend	Local, development, UAT	No forma parte de la operación productiva.

3.1 Flujo recomendado de uso

1. Levantar backend local o ingresar al ambiente UAT autorizado.
2. Abrir /docs desde navegador.
3. Presionar Login con Google.
4. Confirmar que la cuenta existe en Firebase y está habilitada en Firestore.
5. Esperar que el token se aplique automáticamente al esquema HTTPBearer.
6. Ejecutar primero GET /me para confirmar identidad, rol y estado activo.
7. Ejecutar endpoints funcionales según el rol asignado.
8. Registrar evidencia de respuesta, código HTTP y timestamp cuando corresponda.

4. Autenticación y autorización

Los endpoints protegidos requieren encabezado Authorization con esquema Bearer. El token debe ser un ID Token emitido por Firebase Auth para una cuenta válida.

Authorization: Bearer <firebase_id_token>

Rol	Alcance general	Observación de API
sin_acceso	Usuario autenticado pero no habilitado operativamente.	Debe recibir 403 en endpoints protegidos.
vendedor	Opera clientes, interacciones, oportunidades y propuestas propias.	No debe acceder a registros asignados a otros vendedores.
supervisor	Consulta información consolidada y módulos de supervisión.	Puede acceder a dashboards y usuarios con alcance superior al vendedor.
admin	Administra usuarios, roles, importaciones y documentos RAG.	Posee acceso completo a operaciones sensibles.

4.1 Reglas mínimas de seguridad

- Sin Authorization válido, el backend debe responder 401.
- Con token válido pero usuario sin permiso, el backend debe responder 403.
- Con usuario inactivo, el backend debe bloquear la operación aunque el token sea válido.
- Los vendedores deben quedar restringidos por vendedorUid u ownerUid según el recurso.
- Los campos sensibles no deben ser aceptados por payloads no autorizados.
- La API key de proveedores externos no debe exponerse en frontend ni en variables VITE_*

5. Inventario general de endpoints

La siguiente matriz resume los endpoints principales expuestos por el MVP web avanzado y sus roles esperados. La matriz es una guía de prueba y debe leerse junto con los modelos OpenAPI generados por FastAPI.

Área	Método y ruta	Rol mínimo	Uso funcional
Salud	GET /health	Público/controlado	Verifica que la API responde.
Readiness	GET /readiness	Público/controlado	Valida configuración mínima para UAT/go-live sin exponer secretos.
Usuario actual	GET /me	Usuario autenticado	Confirma token, uid, email, rol y estado activo.
Auth	POST /auth/login	Usuario autenticado	Sincroniza usuario autenticado en Firestore.
Auth	POST /auth/register	Usuario autenticado	Registra o actualiza usuario autenticado.
Clientes	GET /clientes	vendedor	Lista clientes visibles por rol y filtros.
Clientes	POST /clientes	vendedor	Crea cliente y asigna vendedor según rol.
Clientes	GET /clientes/{cliente_id}	vendedor	Consulta detalle de cliente visible.
Clientes	PATCH /clientes/{cliente_id}	vendedor	Actualiza cliente visible con validación de propiedad.
Clientes	DELETE /clientes/{cliente_id}	vendedor	Baja lógica de cliente visible.
Clientes CSV	POST /clientes/import-csv	admin	Importación masiva validada de clientes.
Interacciones	GET /interacciones	vendedor	Lista interacciones filtradas por cliente.
Interacciones	POST /interacciones	vendedor	Registra llamada, visita, correo o reunión.
Oportunidades	GET /oportunidades	vendedor	Lista oportunidades visibles por rol.
Oportunidades	POST /oportunidades	vendedor	Crea oportunidad comercial.
Oportunidades	PATCH /oportunidades/{id}	vendedor	Actualiza etapa, valor, probabilidad o descripción.
Oportunidades	GET /oportunidades/{id}/detalle	vendedor	Recupera oportunidad con interacciones y propuestas asociadas.

Área	Método y ruta	Rol mínimo	Uso funcional
Propuestas	GET /propuestas	vendedor	Lista propuestas visibles por rol y filtros.
Propuestas	POST /propuestas	vendedor	Crea propuesta vinculada a oportunidad.
Propuestas	PATCH /propuestas/{id}	vendedor	Actualiza estado, monto o notas.
Dashboard	GET /dashboard/vendedor	vendedor	Métricas propias del vendedor autenticado.
Dashboard	GET /dashboard/supervisor	supervisor	Métricas consolidadas del equipo.
Usuarios	GET /users	supervisor	Lista usuarios por estado.
Usuarios	GET /users/{uid}	supervisor	Consulta perfil operativo de usuario.
Usuarios	PATCH /users/{uid}	admin	Actualiza datos operativos del usuario.
Usuarios	PATCH /users/{uid}/role	admin	Cambia rol operativo.
Usuarios	PATCH /users/{uid}/status	admin	Activa o desactiva usuario.
Preferencias	PATCH /users/me/preferences	autenticado	Actualiza tema o idioma propio.
Competencia	GET /competencia/repository	supervisor	Consulta repositorio de inteligencia competitiva.
RAG	POST /rag/chat	vendedor	Consulta asistente documental con fuentes.
RAG	GET /rag/conversations	vendedor	Lista historial de conversaciones según alcance.
RAG Admin	POST /rag/documents/upload	admin	Carga e indexa documento interno.
RAG Admin	POST /rag/documents/reindex	admin	Reconstruye índice semántico.
RAG Admin	GET /rag/documents	admin	Lista documentos disponibles en el corpus.

6. Contratos de entrada y salida

Los contratos formales se derivan de los modelos Pydantic publicados por OpenAPI. Swagger permite inspeccionar los schemas, campos obligatorios, tipos, límites y respuestas esperadas. Esta sección resume los contratos operativos más relevantes para validación de cliente.

6.1 Clientes e importación CSV

Modelo	Campos principales	Reglas relevantes
ClienteCreate	nombre, empresa, email, telefono, rubro, region, estado, vendedorUid, ownerUid	Rechaza campos extra. Email válido. Teléfono chileno normalizado. Rechaza valores que inicien con fórmula.
ClienteUpdate	Campos parciales de cliente	Solo actualiza campos enviados. No admin no puede reasignar vendedorUid u ownerUid.
ClienteResponse	id, datos de cliente, createdAt, updatedAt	No retorna registros con deletedAt en listados normales.
CsvImportResult	totalFilas, importados, fallidos, errores	Importa solo si no existen errores de validación o duplicidad.

6.2 Flujo comercial

Modelo	Campos principales	Reglas relevantes
InteractionCreate	clienteId, tipo, fecha, resumen, resultado, proximaAccion	tipo permitido: llamada, visita, correo, reunion. Resumen obligatorio.
OpportunityCreate	clienteId, titulo, etapa, valorEstimado, probabilidad, descripcion	etapa permitida: nuevo, contactado, cotizacion, negociacion, ganado, perdido.

Modelo	Campos principales	Reglas relevantes
OpportunityUpdate	titulo, etapa, valorEstimado, probabilidad, descripcion	Actualización parcial, probabilidad entre 0 y 100, valor no negativo.
ProposalCreate	clienteId, oportunidadId, titulo, montoNeto, descuentoPct, estado, notas	Debe estar vinculada a oportunidad. descuentoPct entre 0 y 100.
ProposalResponse	datos base, montoDescuento, montoTotal	El cálculo de descuento y total se realiza server-side.

6.3 Usuarios y permisos

Modelo	Campos principales	Reglas relevantes
UserResponse	uid, email, nombre, rol, cargo, rango, zona, appMovil, webApp, theme, language, activo	Roles normalizados: sin_acceso, admin, supervisor, vendedor.
UserUpdate	nombre, rol, cargo, rango, zona, appMovil, webApp, theme, language, activo	Solo admin actualiza usuarios mediante PATCH /users/{uid}.
UserRoleUpdate	rol	Cambia solo el rol operativo. Rechaza valores no reconocidos.
UserStatusUpdate	activo	Bloquea o habilita acceso funcional del usuario.
UserPreferencesUpdate	theme, language	Permite cambio de preferencias propias.

6.4 RAG parcial

Modelo	Campos principales	Reglas relevantes
RagChatRequest	pregunta, conversacion_id	pregunta obligatoria, no vacía, máximo 1000 caracteres.
RagChatResponse	respuesta, fuentes, conversacion_id, tokens_usados, timestamp, diagnostico, titulo_conversacion	Retorna fuentes cuando existe contexto recuperado.
RagUploadResponse	mensaje, documento, chunks_indexados, disponible_en	Carga aceptada solo para admin. Formatos permitidos: PDF, DOCX y TXT.
RagDocumentResponse	id, nombre, nombreOriginal, chunks_count, subidoPor, activo	Metadatos públicos del corpus, no expone secretos.

Alcance RAG

El módulo RAG se considera parcialmente entregado. Swagger permite validar su estado actual, pero la aceptación no implica mejora de prompts, entrenamiento, tuning, cambio de proveedor, carga masiva de corpus, evaluación avanzada de calidad ni SLA del proveedor externo.

7. Pruebas funcionales desde Swagger

La prueba mínima debe ejecutarse por rol, usando cuentas reales o cuentas de prueba autorizadas por el cliente. Las pruebas deben registrar endpoint, payload, código HTTP, resultado esperado y evidencia visual o JSON de respuesta.

Código	Caso	Rol	Resultado esperado
API-01	GET /health sin token	Sin sesión	200 OK con status ok.
API-02	GET /me sin token	Sin sesión	401 no autorizado.
API-03	Login Google en /docs	Cuenta válida	Token aplicado a HTTPBearer.
API-04	GET /me con token	vendedor	Respuesta con uid, email, rol y activo.
API-05	GET /clientes	vendedor	Lista solo clientes propios.

Código	Caso	Rol	Resultado esperado
API-06	POST /clientes	vendedor	Cliente creado con vendedorUid del usuario.
API-07	GET /clientes/{id} ajeno	vendedor	403 por restricción de propiedad.
API-08	POST /interacciones	vendedor	Interacción vinculada a cliente visible.
API-09	POST /oportunidades	vendedor	Oportunidad creada en etapa inicial.
API-10	POST /propuestas	vendedor	Propuesta creada con cálculo server-side.
API-11	GET /dashboard/supervisor	supervisor	Métricas consolidadas visibles.
API-12	PATCH /users/{uid}/role	admin	Rol actualizado y usuario reflejado en Firestore.
API-13	POST /clientes/import-csv	admin	Importación aceptada o errores detallados por fila.
API-14	POST /rag/chat	vendedor	Respuesta con fuentes o respuesta local sin contexto.
API-15	POST /rag/documents/upload	admin	Documento validado, guardado e indexado.

7.1 Secuencia recomendada para demostración

1. Validar disponibilidad con /health y /readiness.
2. Autenticarse con Google desde la consola /docs.
3. Ejecutar /me para confirmar identidad y rol.
4. Probar un flujo comercial completo: cliente, interacción, oportunidad y propuesta.
5. Probar dashboard de vendedor o supervisor según rol.
6. Probar control negativo: endpoint restringido con rol insuficiente.
7. Probar RAG con una consulta simple y revisar fuentes o mensaje sin contexto.
8. Cerrar sesión y confirmar que las llamadas protegidas sin token fallan.

8. Contrato de errores HTTP

El comportamiento de errores debe ser consistente para facilitar soporte, depuración y validación UAT. La siguiente matriz resume los códigos esperados.

HTTP	Uso esperado	Ejemplo de causa
400	Regla de negocio inválida	CSV sin encabezados o columnas no soportadas.
401	Token ausente, inválido o expirado	No se envía Authorization Bearer.
403	Usuario sin permisos, inactivo o sin acceso a plataforma	Vendedor accede a cliente ajeno o usuario activo=false.
404	Recurso no encontrado	Cliente inexistente, ruta Swagger bloqueada en producción.
409	Conflicto de datos	Email duplicado o conflicto de recurso cuando aplique.
413	Payload demasiado grande	CSV o carga RAG supera tamaño máximo permitido.
422	Payload inválido	Campos extra, enums inválidos, email mal formado.
429	Rate limit o cuota de Firestore	Demasiadas solicitudes o cuota excedida.
503	Servicio no disponible	Firestore o proveedor externo temporalmente no disponible.

8.1 Criterio de respuesta segura

Los errores no deben exponer credenciales, service accounts, API keys, stack traces productivos, valores internos sensibles ni detalles que permitan ampliar superficie de ataque. En desarrollo puede existir mayor detalle controlado para depuración, pero en producción se debe preferir respuesta genérica y trazabilidad interna.

9. Seguridad de la consola Swagger

Swagger es una herramienta técnica de validación. No constituye una pantalla de usuario final y no debe permanecer disponible públicamente en producción.

Control	Criterio
Ambiente	Habilitar solo en local, development o UAT.
Producción	Responder 404 o deshabilitar /docs y /openapi.json.
Autorización	Usar ID Token Firebase; no pegar service accounts ni claves privadas.
Persistencia de token	No persistir autorización indefinidamente. Cerrar sesión al terminar prueba.
CORS	Permitir solo orígenes definidos; no usar wildcard en producción.
Evidencia	Capturar solo datos necesarios, sin registrar tokens completos en informes.

Advertencia

No se debe enviar ni almacenar el ID Token completo en correos, tickets, capturas públicas o documentos de cliente. Para evidencia, basta con indicar usuario, rol, endpoint, estado HTTP y timestamp.

10. Validación técnica previa a entrega

Antes de entregar la API para revisión técnica del cliente, se debe completar una validación mínima de disponibilidad, seguridad, contratos y flujos críticos.

ID	Validación	Resultado esperado
VAL-01	Backend inicia sin errores.	Uvicorn expone API en host y puerto configurados.
VAL-02	GET /health responde.	status ok.
VAL-03	GET /readiness responde.	Reporte sin secretos con estado de configuración.
VAL-04	/docs disponible fuera de producción.	Consola carga y permite login Google.
VAL-05	/docs bloqueado en producción.	404 o inaccesible.
VAL-06	Endpoints protegidos fallan sin token.	401.
VAL-07	Usuario sin rol habilitado falla.	403.
VAL-08	Usuario inactivo falla.	403.
VAL-09	Payload con campo extra falla.	422 cuando el modelo use extra=forbid.
VAL-10	Vendedor no accede a dato ajeno.	403 o 404 según regla aplicada.
VAL-11	CSV inválido entrega detalle.	Errores por fila y sin importación parcial inconsistente.
VAL-12	RAG responde sin exponer secretos.	Respuesta con fuentes o fallback local.

11. Criterios de aceptación

La documentación API / Swagger se considera aceptable para la entrega si permite ejecutar y evidenciar los flujos técnicos principales sin requerir acceso directo a Firestore ni exposición de credenciales sensibles.

Criterio	Condición de aceptación
CA-SWG-01	La consola Swagger carga correctamente en ambiente no productivo autorizado.
CA-SWG-02	El login con Google obtiene ID Token y autoriza llamadas protegidas.
CA-SWG-03	El endpoint /me confirma identidad, rol y estado del usuario.
CA-SWG-04	Los endpoints de clientes, flujo comercial, dashboard y usuarios se prueban según rol.
CA-SWG-05	Los errores 401, 403, 404 y 422 pueden ser provocados y observados en pruebas controladas.
CA-SWG-06	El módulo RAG puede probarse en su estado parcial documentado.
CA-SWG-07	En producción, /docs y /openapi.json no quedan expuestos públicamente.
CA-SWG-08	La evidencia de prueba no contiene tokens completos, API keys ni service accounts.

12. Anexo de comandos y ejemplos

Los siguientes ejemplos se entregan como referencia operativa para validación local. Los valores reales de tokens y dominios deben ser reemplazados por los del ambiente correspondiente.

12.1 Verificación de salud

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/readiness
```

12.2 Prueba de usuario autenticado

```
curl -H "Authorization: Bearer <firebase_id_token>" \
  http://127.0.0.1:8000/me
```

12.3 Listado de clientes

```
curl -H "Authorization: Bearer <firebase_id_token>" \
  "http://127.0.0.1:8000/clientes?limit=100"
```

12.4 Crear interacción comercial

```
POST http://127.0.0.1:8000/interacciones
Headers:
  Authorization: Bearer <firebase_id_token>
  Content-Type: application/json
```

```
Payload:
{
  "clienteId": "<cliente_id>",
  "tipo": "llamada",
  "fecha": "2026-07-06T12:00:00Z",
  "resumen": "Contacto comercial validado."
}
```

12.5 Consulta RAG

```
POST http://127.0.0.1:8000/rag/chat
Headers:
  Authorization: Bearer <firebase_id_token>
  Content-Type: application/json
```

Payload:

```
{  
  "pregunta": "Resume las capacidades comerciales disponibles."  
}
```

Cierre documental

Este documento deja formalizado el uso de Swagger como mecanismo de prueba técnica controlada de la API Enci/ Ventas. Su utilización debe limitarse a validación, soporte técnico y recepción UAT, manteniendo fuera de producción la exposición pública de la consola y del esquema OpenAPI.